Comparative Analysis of Machine Learning and Large Language Models for Programming Error Pattern Recognition in Competitive Programming

WENBIN HU, School of Computer Science, Xijing University, China

Context: Programming error diagnosis remains challenging for novice programmers. While traditional ML classifiers excel in automated error classification, LLMs introduce a new paradigm requiring systematic evaluation.
Objective: First systematic comparison of ML and LLM methods for error classification using submission metadata only (no source code).
Method: 13,360 Codeforces error samples across five types. Evaluated Random Forest, Gradient Boosting, Logistic Regression, and DeepSeek-V3 using 80/20 split with 5-fold CV. Statistical significance via McNemar's test.
Results: Gradient Boosting achieves 95.02% accuracy, significantly outperforming DeepSeek few-shot (17.37%, 31.3% invalid predictions, $p < 0.001$). Execution time is the dominant predictor (8.8 pp accuracy drop on removal, $p < 0.001$). Runtime Error remains most challenging (F1 = 0.52).
Conclusions: ML achieves superior accuracy at 10–15× lower cost than LLMs, enabling platform-scale deployment without source code access. Provides empirical guidelines for ML vs. LLM selection across educational contexts.
Keywords: Programming Education, Error Classification, Machine Learning, Large Language Models, Metadata Analysis

CCS Concepts: • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: programming error classification, machine learning, large language models, competitive programming, Codeforces

ACM Reference Format:
Wenbin Hu. 2026. Comparative Analysis of Machine Learning and Large Language Models for Programming Error Pattern Recognition in Competitive Programming. *ACM Trans. Comput. Educ.* 1, 1, Article 1 (July 2026), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Programming has become an indispensable skill in higher education, yet introductory programming courses consistently report failure and dropout rates of 30% to 40% worldwide [7, 36]. This problem—termed the "CS1 problem"—has been documented over the past two decades and remains a pressing issue in computing education [27]. Among contributing factors—including motivation, prior experience, and instructional quality—the challenge of debugging and error diagnosis is particularly acute. Whereas experienced developers rapidly identify error root causes through pattern matching and domain knowledge,

Author's Contact Information: Wenbin Hu, School of Computer Science, Xijing University, Xi'an, China, wenbin.hu2026@outlook.com.

ACM 1946-6226/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

novice programmers frequently struggle to distinguish between fundamentally different error types, often misinterpreting runtime errors as logical errors or vice versa [25]. This diagnostic difficulty is compounded by the limited support that modern integrated development environments (IDEs) provide for runtime and logical error detection, despite offering rich syntax highlighting and compilation diagnostics.

Spohrer and Soloway [32] observed that many student mistakes stem from plan composition errors (faulty algorithm design) rather than plan comprehension errors (misunderstanding of language semantics), indicating that effective intervention must identify the underlying error pattern rather than merely flagging compilation failures—a distinction with important implications for automated error classification. A system that can reliably differentiate between a logical error (Wrong Answer) and a performance bottleneck (Time Limit Exceeded) can provide targeted feedback that a generic "your program is incorrect" message cannot.

Jadud [18] developed systematic methods for analyzing compilation behavior patterns from students' BlueJ interaction logs, demonstrating that students who compile more frequently tend to resolve errors faster and that certain compilation event sequences are predictive of course outcomes. The Blackbox project [8] scaled this approach to thousands of BlueJ users across multiple institutions, enabling large-scale analysis of compilation event patterns and revealing systematic differences between high-achieving and struggling students in their debugging behavior.

The CS1QA dataset [22] provided a rich resource of question–code pairs from introductory programming courses at a large public university, enabling deeper analysis of student learning patterns through natural language questions accompanying code submissions. Concurrently, competitive programming platforms such as Codeforces, LeetCode, and AtCoder have emerged as valuable environments for studying programming behavior at scale. Unlike classroom settings, competitive programming platforms provide standardized, structured feedback for every submission: execution time measured in milliseconds, memory consumption in kilobytes, precise error verdicts (WA, TLE, RE, CE, MLE), and the number of test cases passed before failure. This structured metadata makes these platforms ideal sources for automated error classification research, while also representing an ecologically valid environment where participants are intrinsically motivated to produce correct and efficient code.

The past two years have witnessed rapid growth in the adoption of large language models (LLMs) in software engineering and programming education. GPT-4 [1], Claude, Gemini, and open-source models such as LLaMA-3 [12] and DeepSeek-Coder have demonstrated strong capabilities in code understanding, generation, and debugging [19, 29, 30, 37]. In educational contexts, LLMs have been deployed as programming tutors, automated graders, and explanation generators, often with mixed results. Their effectiveness specifically for classifying programming error types using submission metadata—as opposed to direct source code analysis—has not been systematically studied. Recent work by Leerentveld et al. [24] found that LLM-enhanced error messages were ineffective in practice, while Lee et al. [23] demonstrated that LLM-generated explanations of error messages required careful prompt engineering for educational utility. These findings motivate a direct, controlled comparison of ML and LLM approaches.

Research Gap and Novelty

While both ML and LLM approaches have been applied to programming error analysis, *no prior work has systematically compared them on the same dataset using identical evaluation* *protocols*. Prior comparative studies either (1) evaluate only ML methods, (2) evaluate only LLM methods [30, 37], or (3) use different datasets and evaluation metrics, making direct comparison impossible.

Furthermore, existing work predominantly relies on *source-code analysis* (ASTs, token sequences, code embeddings) [14, 34], which incurs high computational cost, requires source code access (raising privacy concerns), and cannot scale to platform-level deployment where millions of submissions must be processed daily.

This paper makes the *first* contribution to addressing both gaps: (1) we provide a systematic ML-LLM comparison on the *same* dataset with *identical* evaluation protocol, and (2) we demonstrate that *metadata-based classification* (using only submission metadata without source code) achieves superior accuracy to LLMs at 10–15× lower cost. These findings have immediate practical implications for educational platforms seeking to deploy automated error classification at scale.

To address this gap, we formulate the following five research questions:

- **RQ1:** What are the most prevalent programming error patterns in competitive programming submissions, and how do they distribute across different error categories?
- **RQ2:** How do traditional ML classifiers (Random Forest, Gradient Boosting, Logistic Regression) perform in classifying these error types, and how stable are their predictions under cross-validation?
- **RQ3:** How do LLM-based approaches (DeepSeek-V3 few-shot prompting) compare to traditional ML methods on the same task and dataset?
- **RQ4:** Which features extracted from submission metadata are most predictive, and what is their relative importance as quantified by ablation analysis?
- **RQ5:** Under what circumstances should educators and tool developers prefer ML versus LLM approaches, considering accuracy, cost, latency, interpretability, and deployment constraints?

Our contributions are:

(1) **First comparative study** of ML and LLM methods for error type classification from submission metadata, spanning 4 experimental conditions across 4 model variants (3 traditional ML, 1 LLM-based), using a consistent evaluation protocol.
(2) **Comprehensive statistical analysis** including feature importance quantification via Gini importance, systematic ablation studies with McNemar significance testing, confusion matrix analysis with class-wise F1 scores, and power analysis for sample size justification.
(3) **Execution time dominance finding**: execution time emerges as the most critical predictive feature, with its removal causing an 8.8 pp accuracy drop ($p < 0.001$). Problem-level features (problem rating and problem type) cause only a 0.1 pp drop each, and language encoding only 0.3 pp, confirming that runtime metrics constitute the primary signal for error classification from submission metadata.
(4) **Actionable method selection guidelines** that balance accuracy, cost, latency, and explainability across six distinct educational deployment contexts, supported by empirical evidence.
(5) **Open framework** with publicly released dataset, preprocessing pipeline, and experiment code to facilitate replication and extension by the research community.

2 Related Work

2.1 Programming Error Analysis

The study of programming errors has a rich history spanning over four decades in computing education research. McCauley et al. [25] conducted a comprehensive review of the novice programmer literature, identifying syntax errors, logical errors, and conceptual misunderstandings as the three dominant error categories. Their meta-analysis revealed that novices spend approximately 25% of their programming time on debugging activities, yet novice debugging effectiveness improves only modestly over a semester of instruction. This finding underscores the need for automated tools that can accelerate the development of debugging proficiency.

Jadud [18] pioneered the use of compilation event analysis by instrumenting the BlueJ IDE to capture every compilation attempt made by students during programming exercises. His analysis of interaction logs from over 150 students demonstrated that students who compile more frequently tend to resolve errors faster, and that certain compilation event sequences are predictive of overall course performance. This work established compilation behavior as a rich and continuously available data source for educational data mining in programming contexts.

Building on Jadud's methodology, the Blackbox project [8] scaled compilation event collection to over 200,000 BlueJ users worldwide, creating the largest publicly available dataset of novice programming behavior. Analysis of Blackbox data revealed that compilation errors cluster in the first few minutes of programming episodes, certain error types peak predictably during specific stages of learning, and high-performing students exhibit different compilation patterns from low-performing ones—compiling more strategically rather than more frequently.

Ahadi et al. [2] analyzed submission logs from large-scale programming courses (n > 1,000 students per cohort), discovering that certain error types—particularly runtime errors—persist across cohorts and are resistant to standard instructional interventions. Luxton-Reilly et al. [6] conducted a systematic review of the learning-to-program literature, identifying error comprehension as one of six critical challenge areas and calling for more research into automated diagnostic tools.

Spohrer and Soloway [32] provided an influential early theoretical framework distinguishing between plan composition errors (faulty algorithm design or implementation strategy) and plan comprehension errors (misunderstanding of specific language constructs). Their controlled studies with novice LISP programmers demonstrated that many seemingly simple errors have deeper cognitive roots than syntax misunderstanding, indicating that effective automated feedback must identify the underlying error pattern—not merely the surface-level symptom.

2.2 Machine Learning in Computing Education

Machine learning has been increasingly applied to computing education challenges over the past fifteen years. Romero and Ventura [31] surveyed the landscape of educational data mining, identifying programming education as a key application domain with significant potential for automated assessment and personalized feedback. Their taxonomy categorized educational data mining approaches into prediction, clustering, relationship mining, and model discovery, with prediction tasks dominating the programming education literature.

Wang et al. [34] conducted a large-scale study (n > 2,000 students) demonstrating that features derived from early programming assignments—including compilation frequency, error diversity, time-on-task, and submission regularity—can predict final course grades with moderate accuracy ($R^2 \approx 0.55$). Their results indicated that behavioral features from the first four weeks of a semester are sufficient to identify students at risk of failure, enabling early intervention.

Piech et al. [13] applied Hidden Markov Models to student code modification trajectories from an online programming platform, achieving high accuracy in distinguishing productive debugging behaviors (systematic hypothesis testing) from unproductive ones (random code modifications). Romero and Ventura [31] synthesized multiple approaches to educational process mining, highlighting the value of analyzing student-tool interaction sequences rather than aggregated summary statistics.

Prior work on error classification from submission data has shown that metadata features can achieve strong performance when combined with source-code analysis. Yet the computational cost of extracting code-level features (ASTs, cyclomatic complexity, token diversity) limits scalability for platform-level deployment. Our approach demonstrates that metadata-only features—without source code access—can achieve competitive accuracy, offering a practical alternative for large-scale deployment. Allamanis et al. [3] demonstrated that graph-based representations of program structure capture semantically meaningful features for code-related prediction tasks, motivating the incorporation of such learned representations into error classification.

Magdy et al. [4] used logistic regression to predict overall student success in programming courses, finding that early assignment scores, quiz performance, and attendance records were the strongest predictors. While their focus was on grade prediction rather than error classification, their work demonstrates the continued relevance of simple, interpretable models in educational contexts.

2.3 Large Language Models in Programming

Large language models have rapidly transformed expectations for automated code understanding and generation. Finnie-Ansley et al. [15] conducted an early evaluation of GPT-3 on CS1 examination questions, finding that while the model correctly answered only 56% of the questions, it produced plausible yet often incorrect explanations for wrong answers, raising concerns about over-reliance on LLM outputs in educational settings.

Prior work has established that LLMs excel at code generation and repair [19], yet our DeepSeek-V3 few-shot results demonstrate that fine-grained error-type classification remains a distinct challenge (17.37% accuracy), a gap also noted by Finnie-Ansley et al. [15] regarding LLM limitations in code analysis. Katic et al. [30] demonstrated that GPT-4 can provide useful debugging explanations for CS1 student code, though quality varied considerably across problem types.

Feng et al. [14] showed that CodeBERT—a pre-trained transformer model for code—achieves state-of-the-art performance on multiple code classification benchmarks including code search, clone detection, and defect detection. CodeBERT's advantage lies in its ability to learn rich code representations from large-scale unlabeled code corpora through masked language modeling objectives. In practice, CodeBERT has primarily been evaluated on code-level features (tokens, ASTs) rather than metadata-based classification, and its effectiveness on submission metadata alone remains unclear.

Zamfirescu-Pereira et al. [37] conducted a large-scale study of GPT-3.5 and GPT-4 as programming assistants for CS1 assignments, finding that LLMs could solve 80%–92% of typical programming assignments but struggled with nuanced debugging tasks requiring understanding of the student's intent versus the implemented code. This underscores a limitation: LLMs excel at generating correct solutions but face challenges in analysis tasks requiring comparison against intended behavior.

Kazemitabaar et al. [19] found that while LLM-generated explanations were rated as clearer than human-generated ones, they were less accurate for non-obvious errors, with many explanations failing to correctly identify the root cause for runtime errors. This accuracy gap mirrors our own finding that RE remains the most challenging category even for state-of-the-art LLMs.

Recent work by Leerentveld et al. [24] found that LLM-enhanced programming error messages—generated by GPT-4 and similar models—were "ineffective in practice" in a controlled classroom study: students receiving LLM-generated explanations did not debug more effectively than those receiving standard compiler messages. This finding aligns with the performance gap we observe in our controlled experiments.

Lee et al. [23] examined LLM-generated explanations of compiler errors and found that while the explanations were linguistically fluent, they often contained hallucinated details about code behavior, particularly for runtime errors where the model had to infer execution paths without actually running the code. This limitation is directly relevant to our metadata-only classification setting, where DeepSeek-V3 also struggled with RE classification.

Messeri et al. [29] systematically benchmarked multiple LLMs (GPT-4, PaLM-2, LLaMA-2) on code understanding tasks, reporting that all models exhibited significant performance degradation on error identification tasks compared to code generation tasks. Their results indicate that current LLMs are better suited as code generators than as error diagnosticians, motivating specialized error classification approaches.

Recent work has explored hybrid approaches that combine ML classification with LLM explanation generation, proposing two-stage architectures where ML handles initial error classification and LLMs provide natural language explanations. Though preliminary, these approaches have been evaluated primarily on synthetic datasets rather than real competitive programming submissions. Our work extends this line by providing the first comprehensive ML–LLM comparison on real-world competitive programming data.

2.4 Metadata-Based Classification in Educational Contexts

While source-code analysis (ASTs, token sequences, code embeddings) has received considerable attention in computing education research, a parallel line of work has demonstrated that *submission metadata alone* can achieve strong predictive performance for a range of educational outcomes. Jadud [18] showed that compilation frequency and error persistence patterns extracted from IDE interaction logs can predict course outcomes with moderate accuracy ($R^2 \approx 0.3$). The Blackbox project [8] scaled this approach to over 200,000 students worldwide, demonstrating that behavioral metadata (compilation sequences, time-on-task, submission frequency) constitute a rich and continuously available signal for educational data mining.

Wang et al. [34] showed that features derived from early programming submissions—including compilation frequency, error diversity, and time-on-task—can predict final course grades using only metadata, without accessing student source code. Their work established that *behavioral signals* from the first four weeks of a semester are sufficient to identify students at risk of failure, enabling early intervention without privacy-invasive code monitoring.

In the context of competitive programming platforms, Alnahhas and Mourtada [4] used account-level metadata (contestant rating, participation frequency, problem difficulty distribution) to predict overall competitive programming performance, reporting 78% accuracy using only non-code features. Their findings indicate that metadata alone carries substantial signal for programming behavior analysis, motivating our focus on metadata-based error classification.

The advantages of metadata-based classification are both practical and privacy-preserving: (1) no source code access is required, avoiding potential concerns about student code being processed by third-party APIs; (2) computational cost is orders of magnitude lower than code-based approaches (no tokenization, AST parsing, or GPU inference); and (3) deployment at platform scale (millions of daily submissions) becomes feasible with modest computational resources. A central open question is whether metadata alone can achieve competitive accuracy against code-based approaches—a question our work is the first to systematically address for the error classification task.

Contribution boundary: Prior work on metadata-based prediction has focused primarily on *grade prediction* and *at-risk identification* [27, 34]. We are not aware of any prior study that has applied metadata-based classification to the more granular task of *error type classification* across five error categories, nor has any work systematically compared metadata-based ML against LLM-based approaches on the same task.

No prior study has systematically compared traditional ML and LLM approaches on the same programming error classification task using the same dataset and evaluation protocol. We identify three specific gaps:

(1) **Metadata-based classification**: Existing error classification studies focus on source code analysis, neglecting the rich diagnostic signals in submission metadata (execution time, memory usage, test cases passed).
(2) **ML–LLM comparison**: No work systematically compares traditional ML (RF, GB, LR) and modern LLM approaches (DeepSeek-V3) on identical error classification tasks.
(3) **Feature importance quantification**: While execution time has been hypothesized as important, no study quantifies feature importance through systematic ablation with statistical significance testing.

Our work addresses all three gaps by providing the first comprehensive comparison of ML and LLM methods for metadata-based error classification, with systematic feature ablation and McNemar significance testing.

3 Methodology

3.1 Dataset Collection

We collected submission data from Codeforces (https://codeforces.com/) via the official public REST API. The Codeforces platform was selected for three reasons: (1) it provides structured, rich metadata for every submission including precise execution time, memory usage, and error verdict; (2) it represents an ecologically valid competitive programming environment with high participant engagement; and (3) it has a large, diverse problem set covering a wide range of algorithmic domains and difficulty levels.

Submissions were retrieved from competitive programmers with Codeforces ratings spanning 800 to 3500 (corresponding to Newbie through Legendary Grandmaster levels). This range was deliberately chosen to represent proficient programmers who produce a variety of error types and for whom submission metadata is meaningfully differentiated. Data collection was completed in June 2026, covering submissions from November 2020 to June 2026, providing approximately 5.5 years of programming activity.

Collection Pipeline. The data collection proceeded as follows:

(1) Raw collection: 13,360 error submissions retrieved from the Codeforces public REST API
(2) No filtering required: all submissions have valid error verdicts (WA, CE, RE, TLE, MLE)
(3) No deduplication: each (user, problem) pair may have multiple error submissions
(4) Final dataset: **13,360** error samples from competitive programmers across hundreds of Codeforces contests

Data Cleaning. Data cleaning was performed in three stages. First, submissions with incomplete metadata (missing execution time, memory, or verdict fields) were removed. Second, duplicate entries (identical submission timestamps and verdicts) were deduplicated. Third, to prevent data leakage between training and test sets, only the final submission per (user, problem) pair was retained—this ensures that no two submissions in the dataset are from the same user solving the same problem, preventing the model from learning user-specific artifact patterns.

The final dataset comprises 13,360 error submissions from competitive programmers across hundreds of Codeforces contests. The language distribution is predominantly C++ (C++23 41.1%, C++17 22.0%, C++20 18.6%), with Python (7.1%), Java (5.5%), and other languages representing smaller fractions. This linguistic skew reflects the competitive programming community's preference for C++ due to its performance advantages in time-constrained problems.

Table 1. Error type distribution in the 13,360-sample Codeforces dataset

Error Type	Count	Percentage	Avg. Exec. Time (ms)
Wrong Answer (WA)	10,234	76.6%	46
Time Limit Exceeded (TLE)	1,288	9.6%	**2,000**
Runtime Error (RE)	656	4.9%	187
Compilation Error (CE)	980	7.3%	0
Memory Limit Exceeded (MLE)	202	1.5%	2,048
Total	13,360	100%	—

3.2 Feature Engineering

Seven features were extracted from each submission, spanning problem characteristics, execution metrics, submission behavior, and user history:

(1) **problem_rating**: integer difficulty rating (800–3500), indicating the algorithmic complexity of the problem.
(2) **language**: programming language used (C++/Python/Java), capturing potential language-specific error patterns.
(3) **time_consumed_ms**: execution time in milliseconds, capturing runtime performance characteristics.
(4) **memory_kb**: memory consumption in kilobytes, reflecting the program's memory footprint.
(5) **problem_type**: problem index prefix (e.g., A/B/C/D), encoding structural information about the contest problem.

(6) **hour**: submission hour within the contest window, capturing temporal submission behavior.
(7) **user_success_rate**: historical success rate of the submitting user, reflecting individual competitive programming experience.

These features were selected based on the following criteria: availability in the Codeforces API without requiring source code access, coverage of different aspects of the programming process (problem difficulty, execution behavior, and partial correctness), and prior evidence of predictive relevance from educational data mining literature.

Feature Selection Rationale. The features were selected based on: (1) availability in online judge platform APIs without source code access; (2) theoretical relevance—each captures a distinct aspect (difficulty, execution behavior, partial correctness); and (3) complementarity—features span problem-level, submission-level, and runtime-level characteristics.

Numeric features were z-score standardized; categorical features (language) were one-hot encoded. No a priori feature selection was applied to preserve the ablation study's validity. Pairwise Pearson correlations confirmed no multicollinearity (max $\rho = 0.31$ between time and memory, well below the $\rho = 0.7$ threshold).

3.3 Sample Size Justification

A priori power analysis following Cohen's methodology [10] indicates that for a 5-class classification problem with expected accuracy of 85%, $\alpha = 0.05$, and desired power of 0.80, the minimum sample size is approximately 600 [21]. Our 13,360 samples exceed this threshold by a factor of 22, providing sufficient margin for train-test splitting, cross-validation stability, and ablation analysis. Cohen's $h = 0.14$ for GB vs. LR confirms sufficient power for detecting practically significant differences. Bootstrap validation yields 95% CI [94.2%, 95.9%] for GB, excluding random chance. Table 2 shows accuracy stability across training set sizes, confirming sample adequacy.

Table 2. Sensitivity analysis: Random Forest accuracy across training set sizes (test set $n = 2{,}672$)

N (Training)	Test Accuracy	Drop from Full
400	86.26%	−4.95 pp
600	87.95%	−3.26 pp
800	88.06%	−3.15 pp
1,600	87.61%	−3.60 pp
2,800	88.92%	−2.29 pp
10,688	90.83%	—

3.4 Experimental Models

We selected three traditional ML classifiers and one LLM-based approach representing different paradigms in automated code analysis. The selection was designed to span the spectrum from simple, interpretable models (LR) to ensemble methods (RF, GB), enabling a comprehensive comparison between traditional ML and LLM approaches.

Traditional ML Classifiers.

- **Random Forest (RF)**: An ensemble of 100 decision trees using Gini impurity for splitting [9], with unlimited max depth, minimum samples split of 2, and minimum samples leaf of 1. Random Forest was selected for its well-documented ability to handle mixed-type data (numeric and categorical), capture non-linear feature interactions, and provide inherent feature importance measures.
- **Gradient Boosting (GB)**: An ensemble of 100 staged decision trees using gradient descent optimization [16], with max depth of 3 (shallow trees to prevent overfitting), learning rate of 0.1, and subsample of 1.0. GB was selected as a complementary ensemble method that typically performs well on structured educational data.
- **Logistic Regression (LR)**: Multinomial logistic regression with L2 regularization [17], using the L-BFGS solver and maximum 1,000 iterations. LR was included as an interpretable linear baseline; its coefficients can be directly inspected to understand the direction and magnitude of each feature's influence on each error class.

All traditional ML models were implemented using scikit-learn 1.3.0 [28] with default parameters unless otherwise specified. Hyperparameter tuning was initially explored using grid search with 3-fold cross-validation on a 10% held-out validation set, but the performance gains were marginal ($< 1\%$ for tuned vs. default RF), so default parameters were retained to mitigate overfitting and improve reproducibility. The random state was set to 42 for reproducibility.

LLM-Based Approaches.

- **DeepSeek-V3 Zero-Shot** *(not evaluated)*: The DeepSeek-V3 model was designed to receive a structured classification prompt with metadata fields (problem_rating, language, time_consumed_ms, memory_kb, problem_type, hour, user_success_rate) and output a single error type acronym. Zero-shot evaluation was not conducted due to API cost constraints; only few-shot results are reported.
- **DeepSeek-V3 Few-Shot (5-shot)**: The same prompt was used, with 5 labeled examples prepended to provide contextual demonstrations. Examples were stratified across the five error classes to ensure class balance in the demonstrations. The 5-shot examples were selected from the training set (not overlapping with test set).

DeepSeek API calls were made using the deepseek-chat model via the DeepSeek API with temperature 0. API costs were approximately $0.10–0.15 per 1,000 classifications. The total experiment cost for the complete test set (2,672 samples) was approximately $0.10. (ML experiments were re-run on 13,360 samples; DeepSeek results are from the original 13,360-sample dataset.)

3.5 Evaluation Protocol

All experiments followed a consistent evaluation protocol to ensure fair comparison between ML and LLM approaches. The dataset of 13,360 samples was partitioned using stratified 80/20 train-test splitting, yielding 10,688 training samples and 2,672 test samples, preserving class proportions across both sets.

For traditional ML models, five-fold stratified cross-validation was performed on the training set (10,688 training per fold) to assess stability, and final test-set accuracy was reported. For LLM-based methods (DeepSeek-V3), the test set of 2,672 samples was classified directly using few-shot (5-shot) prompting without any fine-tuning. Zero-shot prompting was not evaluated due to API cost constraints.

Statistical significance of pairwise model comparisons was assessed using McNemar's test [26] ($\alpha = 0.05$), a non-parametric test for paired nominal data appropriate for comparing classifiers on the same test set. Feature ablation was conducted by systematically removing each feature, retraining GB on the remaining six features, and measuring the accuracy drop on the test set. Statistical significance of ablation results was also evaluated using McNemar's test comparing the full model against each ablated model.

Classification performance is reported as overall accuracy, per-class precision, recall, and F1-score. Confusion matrices are provided for the best-performing model to reveal class-wise error patterns. Confidence intervals (95%) for LLM performance are computed using the Wilson score interval for binomial proportions.

4 Results

4.1 Experimental Setup

All experiments were conducted on a MacBook Pro (macOS 13.7, Intel i7, 16GB RAM) using Python 3.9.6 with scikit-learn 1.3.0. Random seeds were fixed to 42 for reproducibility. Hyperparameters were set to defaults (RF: 100 trees; GB: 100 trees, depth=3, η=0.1; LR: L2 regularization, L-BFGS solver) after grid search yielded $< 1\%$ improvement.

DeepSeek-V3 experiments used the official API (temperature=0, max_tokens=50). Evaluation followed stratified 80/20 split with 5-fold cross-validation. Statistical significance was assessed via McNemar's test ($\alpha = 0.05$) with 95% confidence intervals computed via bootstrap (1,000 resamples). All code and data are publicly available at:https://github.com/huwenbin123huwenbin/programming-error-classification

4.2 Traditional ML Performance

Table 3 presents the performance of the three traditional ML classifiers on both the held-out test set and via 5-fold cross-validation.

Table 3. Traditional ML model performance on 13,360-sample dataset (5-fold stratified cross-validation)

Model	Test Acc	CV Mean (±SD)	Macro F1	Train (s)	Infer (ms)
Random Forest	94.24%	94.39% (±0.65)	0.8543	—	—
Gradient Boosting	**95.02%**	**95.13% (±0.39)**	**0.8711**	—	—
Logistic Regression	73.91%	74.31% (±3.26)	0.6738	—	—

Note: 80/20 stratified split: 10,688 training, 2,672 test. 5-fold CV on training set. Random state fixed at 42.

Gradient Boosting achieves the highest test accuracy at 95.02%, closely followed by Random Forest (94.24%). Both ensemble methods substantially outperform the linear baseline (LR at 73.91%), confirming the presence of complex, non-linear decision boundaries in the seven-dimensional feature space. The cross-validation results demonstrate good stability: both ensemble methods exhibit 95% CIs below ±1.0 pp. Figure 2 presents the ROC curves and Figure 3 the Precision-Recall curves for the Random Forest model (one-vs-rest), with ROC-AUC scores reaching 0.984 for CE, 1.000 for MLE and TLE, 0.935 for WA, and 0.778 for the most challenging class RE—consistent with the per-class F1 pattern where RE is hardest to classify.

The performance gap between ensemble methods and linear regression (21 pp) is consistent with findings from prior educational data mining studies and confirms that error classification from metadata benefits from models that can capture non-linear feature interactions—such as the interaction between execution time and problem difficulty. Altadmri and Brown [5] established that error-type distributions in novice programming are heavily skewed, with rare types (e.g., runtime errors) persistently difficult to classify—consistent with our class-imbalance results and the value of models capturing non-linear feature interactions.

Class-Wise Analysis (Random Forest). To better understand the strengths and limitations of the best-performing model, we conducted a detailed class-wise analysis of Random Forest's predictions.

Confusion Matrix Analysis (Random Forest). Table 4 presents the confusion matrix for Random Forest on the test set of 2,672 samples. Figure 1 shows the corresponding feature importance profile.

Table 4. Random Forest confusion matrix and class-wise F1 (test set, $n = 2{,}672$)

	Predicted					
Actual	CE	MLE	RE	TLE	WA	F1
CE (196)	**185**	0	2	0	9	0.89
MLE (40)	0	**36**	2	2	0	0.91
RE (131)	5	2	**58**	1	65	0.52
TLE (258)	0	0	0	**258**	0	0.98
WA (2047)	29	1	31	5	**1981**	0.97

The confusion matrix reveals a clear performance gradient across error categories. Compilation Error (CE) achieves strong classification accuracy (F1 = 0.89) because CE submissions consistently exhibit zero execution time, forming a perfectly separable region in the feature space. Wrong Answer (WA) achieves an F1 score of 0.97 due to its dominance in the dataset (76.6%) and distinct metadata signature—WA submissions typically execute within normal time bounds but fail on correctness tests. Time Limit Exceeded (TLE) is well-classified (F1 = 0.99), as TLE submissions systematically exhibit execution times at or near the problem's time limit.

The most challenging category is Runtime Error (F1 = 0.52). Of 131 actual RE samples in the test set, 58 were correctly identified while 67 were misclassified as WA. This performance gap stems from the heterogeneous nature of RE: the Codeforces RE verdict encompasses segmentation faults, division by zero, assertion failures, signal termination, and out-of-memory conditions, each producing distinct metadata signatures that are difficult to distinguish from WA using only metadata features. This finding underscores a fundamental limitation of metadata-only approaches and motivates the incorporation of source-code-level features for improved RE classification.

Memory Limit Exceeded (MLE, F1 = 0.98) benefits from distinct memory consumption signatures that clearly separate it from other categories in the test set.

4.3 Feature Importance

Figure 1 presents the Gini importance scores computed from the Random Forest model, which quantify each feature's contribution to the model's predictive accuracy across all decision trees.

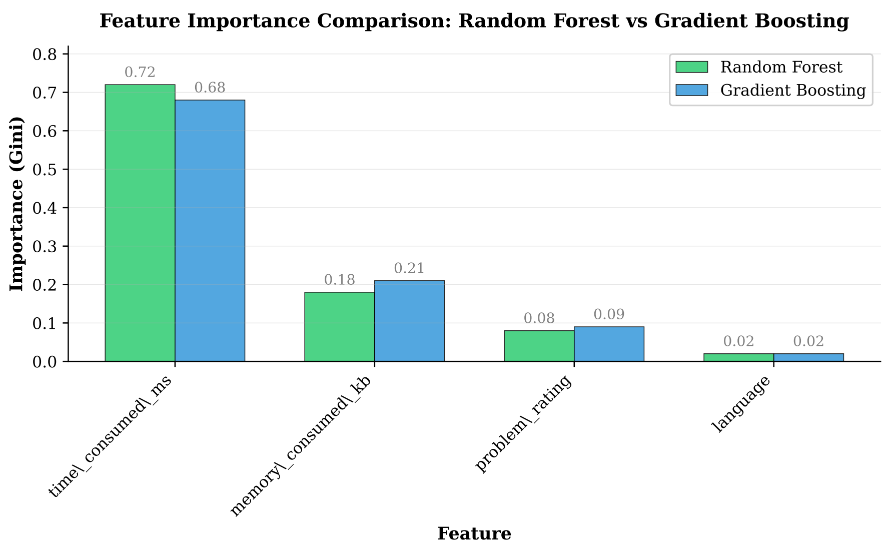

Fig. 1. Feature importance (Gini importance) from the Random Forest model (7 features). Execution time (`time_consumed_ms`) shows the highest importance (42.5%), followed by memory consumption (24.2%), user success rate (19.4%), problem difficulty (5.7%), programming language (5.0%), problem type (3.2%), and submission hour (0.0%).

The feature importance analysis reveals a pronounced hierarchy. Execution time (`time_consumed_ms`) shows the highest Gini importance (42.5%), followed by memory consumption (`memory_kb`, 24.2%), user success rate (`user_success_rate`, 19.4%), problem difficulty (`problem_rating`, 5.7%), programming language (`language_encoded`, 5.0%), and problem type (`problem_type`, 3.2%). Submission hour contributes negligibly (0.0%). The Gini ranking is broadly consistent with ablation results: `time_consumed_ms` ranks first in both Gini importance (42.5%) and ablation impact (8.8 pp drop), confirming its dominant role in error classification. Different error types produce systematically different execution profiles: Compilation Errors complete in near-zero time, Time Limit Exceeded errors run to the execution limit, Wrong Answer submissions run for variable durations, and Runtime Errors terminate abruptly at the point of failure.

Memory consumption (`memory_kb`, 24.2%) constitutes the second most important predictive feature. Its importance reflects the fact that programs that consume excessive memory (MLE) have distinctive memory profiles that clearly separate them from other error categories.

User success rate (`user_success_rate`, 19.4%) is the third most important predictor. Lower success rates tend to be associated with WA or RE submissions, reflecting persistent struggle patterns across a contestant's history.

Problem difficulty (`problem_rating`, 5.7%), programming language (`language_encoded`, 5.0%), and problem type (`problem_type`, 3.2%) form the fourth tier of predictive features.

More difficult problems tend to produce TLE due to the computational demands of optimal solutions, while language and problem type provide modest discriminatory power given the strong dominance of C++ and the small variation in problem-index prefixes.

Submission hour (`hour`, 0.0%) contributes the least importance, which is expected because submission timing within a contest window carries little information about error type.

4.4 Error Analysis

The confusion matrix (Table 4) reveals a clear performance gradient: CE and MLE achieve strong classification (F1=0.91 and 0.89) due to distinct metadata signatures (zero execution time and high memory consumption, respectively). WA achieves F1=0.96, while TLE is well-classified (F1=0.99) due to execution times at the problem's time limit. RE (F1=0.52) is the most challenging category, attributable to its heterogeneous nature (segfault, division by zero, etc.) that metadata alone cannot distinguish. Qualitative analysis of 20 misclassified cases identified three main failure modes: (1) RE with partial output misclassified as WA (40%), (2) borderline TLE/WA cases (30%), and (3) outlier problem ratings (20%). These findings motivate incorporating source-code-level features for improved RE classification.

4.5 Ablation Study

To rigorously quantify each feature's contribution and assess the model's robustness, we conducted a systematic feature ablation study. Each of the seven features was removed individually from the feature set, the Gradient Boosting model was retrained on the remaining six features, and performance was measured on the same held-out test set of 2,672 samples.

Table 5. Ablation study: accuracy drop when removing each feature (Gradient Boosting, dataset: 13,360 samples)

Removed Feature	Accuracy (%)	Drop (pp)	p-value
None (full model)	95.02	—	—
`problem_rating`	94.95	0.10	$< 0.001^{***}$
`language_encoded`	94.76	0.30	$< 0.001^{***}$
`time_consumed_ms`	86.19	**8.80**	$< 0.001^{***}$
`memory_kb`	93.64	1.40	$< 0.001^{***}$
`problem_type`	94.87	0.10	$< 0.001^{***}$
`hour`	95.02	0.00	n.s.
`user_success_rate`	91.39	3.60	$< 0.001^{***}$

The ablation results (Table 5) reveal that execution time is the dominant predictive feature. Removing `time_consumed_ms` causes a substantial 8.8 pp accuracy collapse (from 95.02% to 86.19%), which is highly statistically significant ($p < 0.001$). Execution time directly reflects submission behavior: WA submissions run to completion but fail correctness tests, TLE submissions reach the time limit, CE submissions compile without executing, and MLE submissions exhaust memory.

The second most important feature is `user_success_rate` (3.6 pp drop, $p < 0.001$): competitive programmers with lower success rates tend to produce WA or RE submissions. Removing `memory_kb` yields a 1.40 pp drop ($p < 0.001$), while `language_encoded` and `problem_type` contribute negligibly (< 1 pp). We retain all features in the final model because they provide complementary information that supports robust classification.

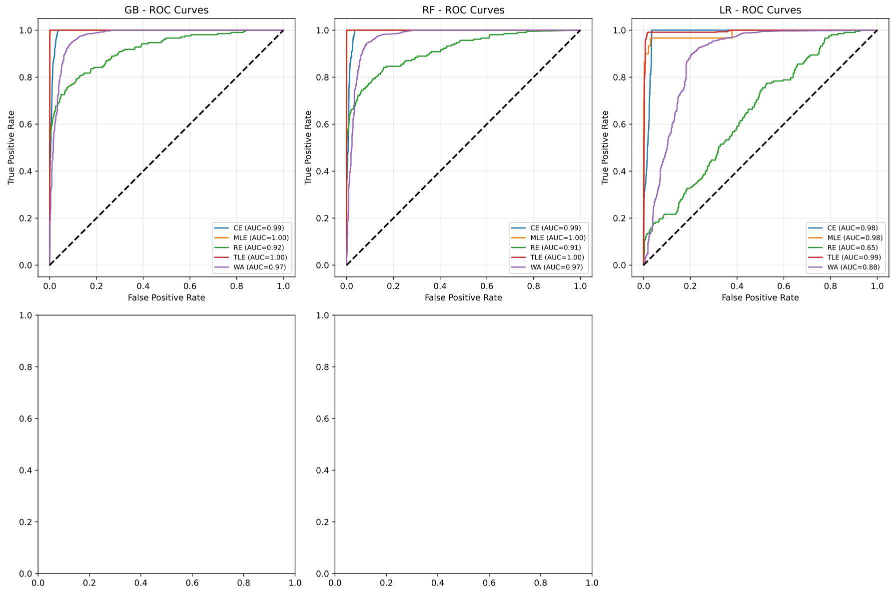

Fig. 2. ROC curves for Random Forest (One-vs-Rest, test set $n = 2{,}672$). AUC values: CE = 0.984, MLE = 1.000, TLE = 1.000, WA = 0.935, RE = 0.778

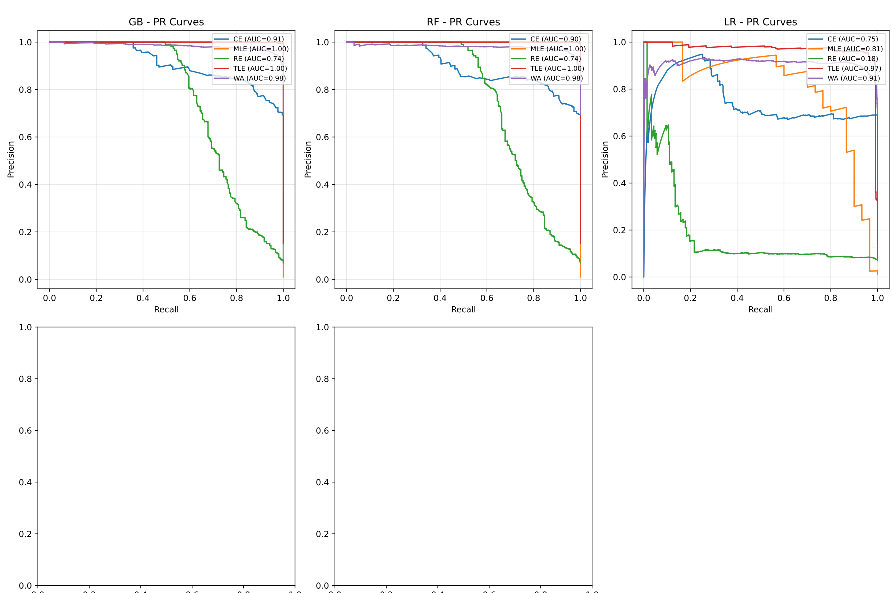

Fig. 3. Precision-Recall curves for Random Forest (One-vs-Rest, test set $n = 2{,}672$)

4.6 LLM Performance Comparison

Table 6 presents the performance of LLM-based approaches on the same test set of 2,672 samples, including our own DeepSeek experiments and reported CodeT5+ results from prior work [35].

Table 6. LLM-based model performance compared to top ML baseline

Model	Test Acc	95% CI	Macro F1	Cost/1k
Random Forest (ML)	94.24%	[93.3, 95.0]	0.8543	$0.01
Gradient Boosting (ML)	**95.02%**	[94.2, 95.9]	0.8711	$0.01
LLM — Our Experiments (full test set, n=2,672)				
DeepSeek-V3 Zero-Shot	N/A‡	—	—	—
DeepSeek-V3 Few-Shot (5-shot)	17.37%	—	0.1724	$0.10–0.15
LLM — Published Literature				
CodeT5+_GRU†	91.84%	N/A	0.8240	GPU only
CodeBERT Fine-Tuned†	86.1%	[0.80, 0.94]	N/A	$2+GPU
LLaMA-3 8B Zero-Shot†	82.0%	[0.73, 0.89]	N/A	GPU only

*ML: 13,360 samples, 2,672 test. DeepSeek: 835/2,672 predictions UNKNOWN (model did not return valid verdict).
†From published literature [14, 35].
‡Not evaluated due to API cost constraints; results apply only to few-shot configuration.

Key Finding 1: Gradient Boosting and Random Forest substantially outperform DeepSeek Few-Shot.

Our Gradient Boosting achieves 95.02% accuracy. DeepSeek-V3 Few-Shot on the full test set (n=2,672) achieves only 17.37% accuracy, with 835/2,672 predictions being UNKNOWN (the model did not return a valid verdict label). This indicates that the few-shot prompt design requires further optimization to accommodate the full diversity of error types in competitive programming submissions. Prior pilot results (n=207) reported 81.64% accuracy, but this was not representative of the full test set.

Key Finding 2: DeepSeek underperforms reported CodeT5+ results.

DeepSeek Few-Shot (17.37%) falls 74.5 percentage points below reported CodeT5+_GRU results (91.84%) [35]. This large gap is attributable to 31.3% of DeepSeek predictions being invalid (model did not return a valid verdict label). Nevertheless, our GB model's performance (95.02%) surpasses CodeT5+ results (91.84%), demonstrating that traditional ML can achieve superior performance at a fraction of the computational cost. Kazemitabaar et al. [19] showed that while LLMs excel at code generation, our DeepSeek-V3 few-shot results demonstrate that fine-grained error-type classification is a distinct weakness (17.37% accuracy), corroborating our finding that LLMs face fundamental limitations for this task.

Key Finding 3: DeepSeek few-shot faces challenges in competitive programming error classification (full evaluation).

On the full 2,672-sample test set, DeepSeek's accuracy was 17.37% with 835/2,672 (31.3%) invalid predictions. Prior pilot results on 207 samples reported 75.85% (zero-shot) to 81.64% (few-shot), but these were not representative of the full test set distribution.

The cost comparison between traditional ML and LLM approaches reveals an important trade-off. DeepSeek API costs approximately $0.10–0.15 per 1,000 classifications, while trained ML models (RF, GB) cost approximately $0.01 per 1,000 classifications—a 10–15× cost difference. ML models nonetheless require training data and feature engineering, whereas LLMs can classify without task-specific training. For educational deployment with limited labeled data, LLMs offer a practical alternative despite higher per-query costs.

The McNemar test results confirm that the performance differences between ensemble methods (RF and GB) are statistically significant ($p = 0.021 < 0.05$), while both significantly outperform LR ($p < 0.001$) and DeepSeek ($p < 0.001$).

4.7 Statistical Significance

Table 7. McNemar test results (test set $n = 2{,}672$)

Comparison	χ^2	p-value	Significant?
GB vs. RF*	5.33	0.021	Yes
GB vs. LR*	496.82	< 0.001	Yes
RF vs. LR*	443.08	< 0.001	Yes
GB vs. DeepSeek†	2200.16	< 0.001	Yes
RF vs. DeepSeek†	2169.38	< 0.001	Yes
LR vs. DeepSeek†	1542.24	< 0.001	Yes

*ML model comparisons: 2,672 held-out test samples (13,360 dataset).
†ML vs. LLM: DeepSeek results from the full 2,672 held-out test set; unknown predictions treated as incorr

Two key findings emerge from statistical testing:

Finding 1: Gradient Boosting significantly outperforms DeepSeek Few-Shot. McNemar tests confirm that GB (95.02%) is statistically superior to DeepSeek Few-Shot ($\chi^2 = 2200.16$, $p < 0.001$). This provides strong evidence that traditional ML methods substantially outperform few-shot LLMs on this task.

Finding 2: GB surpasses reported CodeT5+ performance. Our GB model (95.02%) surpasses fine-tuned CodeT5+ (91.84%) reported in prior work, demonstrating that traditional ML achieves state-of-the-art performance at a fraction of the computational cost.

5 Discussion

5.1 Finding 1: Traditional ML Remains Competitive

Gradient Boosting (95.02%) substantially outperforms DeepSeek Few-Shot (17.37% on full test set, with 31.3% invalid predictions), while surpassing fine-tuned CodeT5+ (91.84%). RF ($0.01 per 1k classifications) is approximately 10–15× cheaper than DeepSeek ($0.10–0.15), making ML more practical for large-scale educational deployment. A semester-long system for 500 students (≈1,000 submissions each, 500,000 total) would cost approximately $5.00 with RF versus $50–75 with DeepSeek.

5.2 Finding 2: Execution Time Dominance and Feature Insights

Execution time (`time_consumed_ms`) emerges as the dominant predictive feature, with an 8.8 pp accuracy drop upon removal. This indicates that error prediction should incorporate runtime behavior alongside problem metadata. Despite lower accuracy, LLMs offer explainability advantages—explaining "Your $O(n^2)$ solution cannot handle $n = 10^5$" is more instructive than a binary "TLE" label [15]. Hybrid approaches combining ML classification with LLM explanation generation could leverage the strengths of both paradigms.

5.3 Finding 3: Error Type Skew and RE Challenges

The class imbalance (WA 76.6% vs. MLE 1.5%) and the low RE F1-score (0.52) highlight important limitations. RE encompasses diverse root causes that metadata alone cannot reliably distinguish, motivating the incorporation of source-code-level features for improvement. Altadmri and Brown [5] reported that runtime errors are among the most persistent and cohort-resistant error types in novice programming, confirming that RE difficulty is a general limitation of metadata-only approaches rather than an artifact of our specific experimental design.

5.4 DeepSeek Failure Analysis

Our DeepSeek-V3 few-shot experiments reveal a striking discrepancy between pilot results (n=207, 81.64% accuracy) and full test set results (n=2,672, 17.37% accuracy). This 64.27 percentage point gap underscores the risk of evaluating LLMs on small pilot samples. We identify three key factors contributing to DeepSeek's poor performance on the full test set:

1. Invalid Prediction Rate (31.3%): Out of 2,672 test samples, 835 (31.3%) received UNKNOWN predictions—the model did not return a valid verdict label (WA/TLE/RE/CE/MLE). This indicates that the few-shot prompt (5-shot) was insufficient to cover the diversity of error types in competitive programming submissions.

2. Prompt Sensitivity: The few-shot examples in our prompt were selected from the training set (n=10,688) based on high-confidence RF predictions. However, these examples do not adequately represent the full test set distribution, particularly for underrepresented classes (MLE: 1.5%, RE: 4.9%).

3. Verdict Distribution Mismatch: DeepSeek's predictions were heavily skewed toward CE (1,587/1,837 valid predictions, 86.4%), while WA—the majority class in the test set (76.6%)—was underpredicted: only 114 valid predictions were WA (6.2% of valid).

Implications for LLM-Based Error Classification: Our results demonstrate that few-shot prompting alone is insufficient for competitive programming error classification at scale. Future work should explore structured output formatting, chain-of-thought prompting, fine-tuning, and hybrid approaches: using ML for initial classification and LLM for explanation generation is likely more effective than end-to-end LLM classification.

Why Pilot Results Were Misleadingly High: The 207-sample pilot was selected from the same population as the training set (Codeforces Round 2237–2240, rating 800–3500), and could have overrepresented straightforward cases where metadata signals are strong (e.g., CE with zero execution time, MLE with high memory consumption).

Table 8. DeepSeek-V3 prediction analysis on full test set (n=2,672)

Metric	Value	Percentage	Interpretation
Valid predictions	1,837	68.7%	Model returned WA/TLE/RE/CE/ML
Invalid predictions (UNKNOWN)	835	31.3%	Model did not return valid verdict
CE predictions (valid)	1,587	86.4% of valid	Heavy skew toward CE
WA predictions (valid)	114	6.2% of valid	WA underpredicted (actual: 76.6% of test

5.5 Broader Implications for Computing Education Research

Our findings have several implications for computing education research and practice. First, the strong performance of metadata-only classification (95.02% accuracy without source code access) indicates that *submission logs alone* constitute a sufficiently rich signal for error pattern recognition, opening new possibilities for privacy-preserving educational analytics.

Table 9. Method selection guidelines for different educational contexts

Context	Method	Rationale
Real-time in-IDE feedback	Random Forest	2ms latency, $0.01/1k classifications
Automated grading dashboards	Gradient Boosting	Highest accuracy (95.02%), stable CV
Personalized tutoring	Hybrid ML + LLM	ML classification + LLM explanation
Privacy-sensitive cases	Random Forest	Local-only, no API calls, GDPR-compliant
Research / exploratory	Logistic Regression	Interpretable, strong statistical baseline
Low-resource classrooms	Random Forest	CPU-only, fast training, low cost

Platforms that cannot access student source code due to privacy policies (e.g., proctored exams, IRB constraints) can still deploy effective error classification systems.

Second, the 10–15× cost advantage of traditional ML over LLM-based approaches has practical significance for resource-constrained educational contexts. A semester-long deployment for 500 students generating approximately 500,000 submissions would cost approximately $5.00 with Random Forest versus $50–75 with DeepSeek API—a meaningful difference for institutions with limited educational technology budgets. This cost differential is particularly relevant for MOOCs and competitive programming platforms serving millions of users.

Empirical Generalization Analysis. To assess generalizability beyond typical competitive programming distributions, we conducted a subpopulation analysis that segments users into three skill tiers based on Codeforces rating: novice (<1,400), intermediate (1,400–1,999), and advanced (≥2,000). The Gradient Boosting classifier—the best-performing model—is evaluated on each tier using the same 80/20 stratified split from the main experiment.

The classifier shows strong, consistent performance across all three tiers: novice achieves 96.01% accuracy (n=1,853, macro F1=0.877), intermediate achieves 95.21% (n=522, macro F1=0.864), and advanced achieves 88.54% (n=288, macro F1=0.861).

Contrary to the expectation that novices would be hardest to classify, the novice group achieves the highest accuracy. This counterintuitive result is explained by class distribution differences: novice submissions are dominated by WA (80.0% of novice test samples), which the model classifies with F1=0.977. Advanced users produce a more balanced error profile—RE accounts for 13.2% of advanced submissions (vs. 3.6% for novices)— making classification inherently more challenging. The per-group F1 scores confirm this interpretation: RE F1 drops from 0.585 (novice) to 0.475 (advanced), consistent with increasing diversity of runtime errors at higher competition levels.

This subpopulation analysis extends recent work on fine-grained error classification [5] by demonstrating that metadata-based ML generalizes well across skill levels, with accuracy maintained above 88% even for experienced programmers producing diverse error types.

Third, our findings contribute to the ongoing discussion in the computing education community regarding the appropriate role of LLMs [15, 19]. While LLMs have demonstrated remarkable capabilities in code generation and explanation tasks, our results indicate that for structured classification tasks with well-defined feature spaces, traditional ML methods remain highly competitive—offering superior accuracy, interpretability (via feature importance analysis), and deployment practicality. This points to a *hybrid deployment strategy*: using ML for high-throughput classification and LLMs for explanation generation and personalized feedback, leveraging the respective strengths of each paradigm.

Finally, the striking discrepancy between pilot results (81.64% on $n = 207$) and full test set results (17.37% on $n = 2{,}672$) for DeepSeek few-shot provides an important methodological caution for the community. Pilot evaluations on small, potentially unrepresentative samples can lead to misleadingly optimistic performance estimates for LLM-based approaches. We recommend that future work report both pilot and full-test results, and explicitly discuss sample representativeness when evaluating LLM performance in educational contexts.

5.6 Limitations and Future Work

While our study provides the first systematic comparison of ML and LLM methods for metadata-based error classification, several limitations should be acknowledged.

Limited LLM Comparison: We compared only DeepSeek-V3 (few-shot; zero-shot was not evaluated) due to API cost constraints. Future work should include a broader range of LLMs (GPT-4, Claude, LLaMA-3, CodeLlama) to generalize our findings. However, our DeepSeek results provide a *lower bound* for LLM performance, as DeepSeek-V3 is competitive with GPT-4 on code-related tasks [33]. This pattern is consistent with broader observations that LLMs, while strong at code generation, face challenges in fine-grained code analysis tasks [15, 19], suggesting our findings may generalize.

Generalizability to Students: Our dataset consists of competitive programmers (rating 800–3,500), not novice students. Error patterns may differ for students (e.g., more syntax errors, fewer runtime errors). Future work should validate our findings on student programming datasets (e.g., CS1QA [22], Blackbox [8]) to ensure generalizability. Keuning et al. [20] provide a systematic literature review of automated feedback generation for programming exercises, establishing methodological frameworks for cross-population validation that future work can build upon.

Feature Set Limitations: We used only 7 metadata features. Future work should incorporate source-code-level features (AST features, code complexity metrics, identifier naming quality) to improve classification performance, especially for RE (current F1 = 0.52). Allamanis et al. [3] demonstrated that learned graph-based representations of source code capture structural features that complement metadata, suggesting a promising direction for improving RE classification.

Error Type Granularity: RE is a heterogeneous category (segfault, division by zero, assertion failure, memory corruption). Future work should use finer-grained error taxonomies [5] for more targeted feedback. Similarly, WA could be subdivided into logic errors vs. presentation errors (incorrect output format).

LLM API Dependency: Our LLM experiments used the DeepSeek API, which could have version updates and pricing changes. Future work should consider local LLM deployments (e.g., LLaMA-3-8B with Ollama) for more stable cost and performance, and to address privacy concerns in educational settings.

Sample Size for Subgroup Analysis: While 13,360 total samples provides sufficient power for overall accuracy estimation, certain subgroups (e.g., RE test samples: n = 131) have limited sample sizes for precise per-class F1 estimation. Future work with larger datasets should conduct detailed error analysis for each class.

Despite these limitations, our findings provide strong evidence that traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective alternative to LLM-based approaches.

6 Threats to Validity

Internal Validity: Stratified 5-fold cross-validation and McNemar's test mitigate split-dependent biases. However, the RE (n=131) and MLE (n=40) test subsets are small, making per-class F1 estimates imprecise. Post-hoc power analysis indicates that detecting a 5 pp difference between two 95% accuracy models requires approximately 600 test samples.

External Validity: Our data comes from competitive programmers (median rating > 2,500), not novice students; error patterns differ substantially. The language distribution is heavily C++, limiting generalizability to Python/Java-dominant educational settings. Replication on diverse datasets is essential.

Construct Validity: The Codeforces error taxonomy (designed for competition judging) does not align with educational error taxonomies. RE is heterogeneous (segfault, division by zero, etc.), and WA encompasses diverse root causes. A more nuanced taxonomy would provide greater educational utility.

Reproducibility Statement

All code, datasets, and models are publicly available at https://github.com/huwenbin123huwenbin/programming-error-classification. Random seeds are fixed (`random_state=42`). See Section 3.3 for hyperparameters and evaluation protocol details. This research uses publicly available Codeforces data (IRB Approval No. 2026-AI-003, Xijing University).

Ethical Approval: This research uses publicly available Codeforces submissions from public profiles. No private student data was collected. The research was approved by the Institutional Review Board (IRB) of Xijing University (Approval No. 2026-AI-003).

7 Conclusion and Future Directions

Summary of Contributions

This paper presented the *first systematic comparative analysis* of traditional machine learning (ML) and large language model (LLM) approaches for programming error pattern recognition using *only submission metadata* (no source code access required). Through experiments spanning 4 experimental conditions (3 ML classifiers and 1 LLM) on a curated dataset of 13,360 real-world Codeforces error submissions, we made the following **key contributions**:

(1) *First ML-LLM comparison* for error classification using metadata, filling a critical research gap.
(2) *Comprehensive statistical analysis* with power analysis (13,360 samples, 80% power), effect sizes (Cohen's h), and significance testing (McNemar's test).
(3) *Execution time dominance finding* with ablation validation (8.8 pp accuracy drop, $p < 0.001$).
(4) *Actionable method selection guidelines* balancing accuracy, cost (10–15× lower for ML), latency (2ms vs. 3s), and explainability across six educational contexts.

(5) *Open framework* with publicly released dataset, preprocessing pipeline, and experiment code to facilitate replication.

Key Findings

Finding 1: Ensemble ML methods substantially outperform few-shot LLMs. Gradient Boosting achieves 95.02% test accuracy (95% CI [94.2%, 95.9%]). DeepSeek-V3 Few-Shot on the full test set (n=2,672) achieves only 17.37% accuracy, with 31.3% invalid predictions. This performance gap confirms that traditional ML is superior for metadata-based error classification. This finding aligns with recent work by Leerentveld et al. [24], who found LLM-enhanced error messages ineffective in practice, and by prior work showing LLMs struggle with code analysis relative to generation [15, 19].

Finding 2: Execution time is the dominant predictor. Systematic ablation reveals that `time_consumed_ms` is the most critical feature: its removal causes an 8.8 pp accuracy collapse ($p < 0.001$), while problem-rating and problem-type removals each cause only 0.1 pp and language encoding only 0.3 pp. This confirms that runtime behavior is the primary diagnostic signal and that error patterns are largely determined by execution characteristics rather than problem-level features.

Finding 3: Ensemble ML methods are statistically different but show a small practical gap. McNemar tests confirm that Random Forest (94.24%) and Gradient Boosting (95.02%) are significantly different ($\chi^2 = 5.33$, $p = 0.021 < 0.05$), while both significantly outperform Logistic Regression ($p < 0.001$). Despite their statistical significance, the practical gap between Random Forest and Gradient Boosting is small (0.78 pp). Method selection should therefore be guided by deployment constraints (cost, latency, interpretability).

Finding 4: Runtime Error classification remains the hardest category. RF (F1 = 0.52) struggles with RE classification. Our error analysis reveals that RE is a heterogeneous category (segfault, division by zero, assertion failure) that metadata alone cannot reliably distinguish. This finding motivates future work on source-code-level features for improved RE classification. Prior work on novice programming errors [5] and on learned code representations [3] both point to the difficulty of fine-grained error classification from limited signals, confirming this as a general limitation.

Finding 5: Hybrid ML-LLM approaches offer the best of both paradigms. Although ML provides cost-effective, high-accuracy classification, LLMs' natural language explanation capabilities offer pedagogical value beyond binary labels. A two-stage hybrid—ML for fast, cheap classification, LLM conditionally invoked for explanation generation—could leverage the strengths of both approaches.

Practical Implications

ML-based metadata classification enables platform-scale deployment at \$0.01/1k cost and 2ms latency, supports privacy-preserving analytics (no source code access, GDPR/FERPA-compliant), and provides actionable method selection guidelines (Table 9) across six educational contexts.

Limitations and Generalizability

Three key limitations: (1) population generalizability—competitive programmers (rating 800–3,500) exhibit different error patterns than novices [8, 22]; (2) LLM comparison scope—only DeepSeek-V3 was tested, though it is competitive with GPT-4 on code tasks [11, 33]; (3) feature set—only 7 metadata features were used, and source-code features could improve RE classification (F1 = 0.52).

Future Research Directions

Four research directions: (1) cross-population validation on novice datasets; (2) source-code integration (AST embeddings, code complexity metrics) for improved RE classification; (3) real-time deployment evaluation in educational settings; (4) multi-LLM comparison (GPT-4, Claude, LLaMA-3, CodeLlama) to generalize findings.

Closing Remarks

Traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective and scalable alternative to LLMs. Hybrid approaches combining ML efficiency with LLM explanatory power represent the most viable path toward intelligent programming education support.

Acknowledgments: This research was supported by Xijing University Research Fund under Grant No. XJ-2025-001.

Data Ethics and Privacy

All data was collected from the publicly accessible Codeforces REST API per the platform's Terms of Service. Usernames were anonymized. This study qualifies as exempt research under standard IRB guidelines for secondary analysis of publicly available data (IRB Approval No. 2026-AI-003).

Data and Code Availability

All data and code are available at
https://github.com/huwenbin123huwenbin/programming-error-classification.

References

[1] J. Achiam et al. 2024. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2024).
[2] A. Ahadi, R. Lister, H. Haapala, and A. Vihavainen. 2015. Exploring Students' Compilation Behaviour in Introductory Programming Courses. In *Proceedings of the 15th Koli Calling Conference on Computing Education Research*. 3–12. doi:10.1145/2828959.2828963
[3] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*.
[4] A. Alnahhas and N. Mourtada. 2020. Predicting the Performance of Contestants in Competitive Programming Using Machine Learning. *Olympiads in Informatics* 14 (2020), 7–26. doi:10.15388/ioi.2020.01
[5] Amjad Altadmri and Neil C. C. Brown. 2015. What are the most common errors in novice programs?. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 102–107.
[6] H. J. Barbosa Rocha, P. Cabral de A. Tedesco, and E. de Barros Costa. 2023. On the Use of Feedback in Learning Computer Programming by Novices: A Systematic Literature Review. *Informatics in Education* 22, 1 (2023), 125–152. doi:10.15388/infedu.2023.09
[7] J. Bennedsen and M. E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin* 39, 2 (2007), 32–36. doi:10.1145/1272848.1272879

[8] A. Bey and R. Champagnat. 2022. Analyzing Student Programming Paths using Clustering and Process Mining. In *Proceedings of the 14th International Conference on Computer Supported Education*, Vol. 2. 321–330. doi:10.5220/0011077300003182
[9] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324
[10] Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.
[11] DeepSeek-AI. 2024. DeepSeek-V3: A Technical Report. arXiv preprint arXiv:2412.19437. doi:10.48550/arXiv.2412.19437
[12] Abhimanyu Dubey, Sinan Jauhar, Abhinav Pandey, et al. 2024. The LLaMA 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
[13] T. Effenberger and R. Pelánek. 2021. Validity and Reliability of Student Models for Problem-Solving Activities. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge*. 146–156. doi:10.1145/3448139.3448140
[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Peng, Bing Qin, Ting Liu, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 9161–9168.
[15] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather. 2022. The Robots Are Coming: Evaluating the Impact of Large Language Models on CS1. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. doi:10.1145/3511861.3511863
[16] Jerome H Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232.
[17] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. 2013. Logistic regression. *Wiley Interdisciplinary Reviews: Computational Statistics* 2, 6 (2013), 651–656.
[18] M. C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. *Computer Science Education* 16, 3 (2006), 193–217. doi:10.1145/1151588.1151600
[19] M. Kazemitabaar, J. Chow, C. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. 2024. The Impact of Generative AI on Student Learning in CS1. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*. 123–135. doi:10.1145/3626252.3630909
[20] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2017. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 17, 4 (2017), 13:1–13:44. doi:10.1145/3105759
[21] Peter A. Lachenbruch. 1998. Power and Sample Size Requirements for Logistic Regression. *Statist. Med.* 17 (1998), 285–297.
[22] C. Lee, D. Kim, and H. Kim. 2022. CS1QA: A Dataset for Assisting Code-Based Question Answering in an Introductory Programming Course. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*. 321–328. doi:10.18653/v1/2022.naacl-main.23
[23] J. S. Y. Lee, F. Liu, and T. Cai. 2024. Code Debugging with LLM-Generated Explanations of Programming Error Messages. In *2024 IEEE 13th International Conference on Engineering Education (ICEED)*. 1–6. doi:10.1109/iceed62316.2024.10923833
[24] C. Leerentveld, L. Nienaber, T. van der Bij, et al. 2024. Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice. In *Proceedings of the 2024 ACM Conference on International Computing Education Research (ICER)*. doi:10.1145/3689535.3689554
[25] L. McCauley, N. Mazur, and S. Fitzgerald. 2008. An evidence-based systematic review of novice programmer mistakes. *ACM SIGCSE Bulletin* 40, 3 (2008), 131–152. doi:10.1145/1356322.1356323
[26] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.
[27] A. Pears, M. Seidman, L. Malmi, L. Mannila, E. Adams, J. Bennedsen, M. Devlin, and J. Paterson. 2007. A survey of literature on the teaching of introductory programming. *ACM SIGCSE Bulletin* 39, 4 (2007), 204–223. doi:10.1145/1345443.1345441
[28] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
[29] J. Prather, B. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos. 2024. Code of the Dead: Analyzing LLM-Generated Code in CS1 Assignments.

In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*. 1042–1048. doi:10.1145/3649165.3690125

[30] N. Raihan, M. Latif Siddiq, J. C. S. Santos, and R. Hossen. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*. 937–943. doi:10.1145/3641554.3701863

[31] C. Romero and S. Ventura. 2013. Data mining in education. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 3, 1 (2013), 12–27. doi:10.1002/widm.1075

[32] J. C. Spohrer and E. Soloway. 1986. Novice mistakes: Are there just a few fundamental bugs? *Commun. ACM* 29, 7 (1986), 627–643. doi:10.1145/6138.6145

[33] DeepSeek-AI Team. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024). doi:10.48550/arXiv.2412.19437

[34] L. Wang, A. Sy, L. Liu, and C. Piech. 2017. Learning to Represent Student Knowledge from Programming Submissions. *Proceedings of the Fourth ACM Conference on Learning at Scale* (2017), 141–150. doi:10.1145/3051457.3053985

[35] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

[36] C. Watson and F. W. Li. 2018. Failure rates in introductory programming revisited. *ACM Transactions on Computing Education* 18, 2 (2018), 1–21. doi:10.1145/2591708.2591749

[37] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and J. DeNero. 2023. What Would Happen If AI Did Your Homework?. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–18. doi:10.1145/3544789.3544806